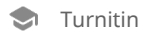


Author

client.c.pdf



Document Details

7 Pages

1,082 Words

6,029 Characters

File Name

unknown_filename

File Size

66.3 KB

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **12 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 13%  Internet sources
- 7%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 12 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 13% Internet sources
- 7% Publications
- 0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
		gitea.iitdh.ac.in 3%
2	Internet	
		forums.freebsd.org 3%
3	Internet	
		review.linaro.org 2%
4	Internet	
		en.wikibooks.org 1%
5	Internet	
		d0nut.medium.com 1%
6	Internet	
		portland.source.codeaurora.org 1%
7	Internet	
		ftp.tvdr.de <1%
8	Internet	
		git.uibk.ac.at <1%

// OS Assignment - dice game - client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <pthread.h>
#include <errno.h>
#include <time.h>
```

// Game settings: Max 3 players, race to R20, each player uses unique FIFO path

```
#define MP 3 // MP = Max Player
#define WC 20 // WC = Winning Criteria / condition
#define FIFO_PREFIX "/tmp/player_"
```

// Shared memory structure

```
typedef struct
{
    int PP[MP]; // PP = player position
    int CT; // CT = current turn
    int game_active;
    int CP; // CP = connected player
    int player_active[MP];
    int FW; // FW = final winner
    int round; // round = number of current round
    char PN[MP][50]; // PN = player names/ client name
    int winner[MP];
    pthread_mutex_t shm_lock; // shm_lock = shared memory lock
    pthread_mutex_t table_sync; // table_sync = Syncing the table/grid display
} GameInfo; // GameInfo = Game information / Game State
```

GameInfo *g_ptr = NULL; // g_ptr = game pointer

```
int player_id = -1;
char player_name[50];
```

// pre-declaration of functions

```
void ssm(); // ssm = setup shared memory
void c_fifos(); // c_fifos = create fifos
void d_grid(const char *last_action, const char *current_status); // d_grid = display grid
```

```
void play_game();
void cleanup();
int fas(); // fas = find available slot
```

```
int main(int argc, char *argv[]) {
    if (argc != 2)
    {
        printf("Please enter a valid instructions: %s (YourName)\n", argv[0]);
        return 1;
    }

    strncpy(player_name, argv[1], sizeof(player_name) - 1);
    player_name[sizeof(player_name) - 1] = '\0';

    printf("=== Dice Race Game - %-20s ===\n", player_name);

    ssm();

    player_id = fas();
    if (player_id == -1)
    {
        fprintf(stderr, "ERROR: Game is full (3 players maximum)\n");
        cleanup();
        return 1;
    }

    // assigning player to the particular slot
    printf("Assigned to Player slot %d\n", player_id + 1);
    pthread_mutex_lock(&g_ptr->shm_lock);
    strncpy(g_ptr->PN[player_id], player_name, 49);
    g_ptr->PN[player_id][49] = '\0';
    pthread_mutex_unlock(&g_ptr->shm_lock);

    c_fifos();
    printf("Connected to server as %s!\n", player_name);
    printf("Waiting for other players to join...\n\n");

    char join_msg[100];
    snprintf(join_msg, sizeof(join_msg), "%s Joined Game.", player_name);

    int last_count = -1;
    while (!g_ptr->game_active)
```

```

{
    int current_count = g_ptr->CP;
    if (current_count != last_count)
    {
        last_count = current_count;
        printf("Players connected: %d/%d\n", current_count, MP);
        d_grid(join_msg, "Waiting for players to join...");
    }
    sleep(1);
}

printf("\nGame Started! Race to %d!\n", WC);

play_game();

d_grid("GAME OVER!", "Final Results");

char filename[256];
snprintf(filename, sizeof(filename), "result_%.s.txt", player_name);

FILE *f = fopen(filename, "w");
if (f)
{
    fprintf(f, "Game Result for %.s\n", player_name);
    fprintf(f, "Winner: %.s\n", g_ptr->PN[g_ptr->FW]);
    fprintf(f, "My Wins: %d\n", g_ptr->winner[player_id]);
    fprintf(f, "Final Position: R%d\n", g_ptr->PP[player_id]);
    fclose(f);
}

printf("\n#####\n");
if (g_ptr->FW == player_id)
{
    printf("    CONGRATULATIONS! YOU WON!    \n");
} else
{
    printf("    Game Over! %.s won the game!    \n",
        g_ptr->PN[g_ptr->FW]);
}
printf("#####\n");

```

```
printf(" Total Wins for %s: %d\n", player_name, g_ptr->winner[player_id]);
printf("-----\n");
```

```
cleanup();
return 0;
```

```
}
```

```
void ssm()
```

```
{
```

```
int shm_fd = shm_open("/dice_game_shm", O_RDWR, 0666);
```

```
if (shm_fd == -1)
```

```
{
```

```
    fprintf(stderr, "ERROR: Could not connect to server: %s\n", strerror(errno));
```

```
    exit(1);
```

```
}
```

```
g_ptr = mmap(NULL, sizeof(GameInfo), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
```

```
if (g_ptr == MAP_FAILED)
```

```
{
```

```
    perror("mmap failed");
```

```
    exit(1);
```

```
}
```

```
close(shm_fd);
```

```
}
```

```
int fas()
```

```
{
```

```
pthread_mutex_lock(&g_ptr->shm_lock);
```

```
int slot = -1;
```

```
for (int i = 0; i < MP; i++)
```

```
{
```

```
    if (!g_ptr->player_active[i])
```

```
    {
```

```
        slot = i;
```

```
        break;
```

```
    }
```

```
}
```

```
pthread_mutex_unlock(&g_ptr->shm_lock);
```

```
return slot;
```

```
}
```

```
void c_fifos()
{
    char fifo_name[256];
    snprintf(fifo_name, sizeof(fifo_name), "%s%d_to_server", FIFO_PREFIX, player_id);
    if (mkfifo(fifo_name, 0666) == -1 && errno != EEXIST) perror("mkfifo");
    snprintf(fifo_name, sizeof(fifo_name), "%s%d_from_server", FIFO_PREFIX, player_id);
    if (mkfifo(fifo_name, 0666) == -1 && errno != EEXIST) perror("mkfifo");
}
```

```
void d_grid(const char *last_action, const char *current_status)
{
    printf("\033[H\033[J");
    // 1. Header
    printf("=====\n");
    printf("    DICE RACE - ROUND %d\n", g_ptr->round);
    printf("=====\n");
    // 2. The Grid
    for (int row = WC; row >= 1; row--)
    {
        printf("|");
        for (int p = 0; p < MP; p++)
        {
            char token = ' ';
            if (g_ptr->player_active[p] &&
                g_ptr->PP[p] == row)
            {
                token = g_ptr->PN[p][0];
            }
            printf(" %c |", token);
        }
        printf(" R%-2d\n", row);
        printf("-----\n");
    }
    // 3. Start Line
    printf("|");
    for (int p = 0; p < MP; p++)
    {
        char token = ' ';
        if (g_ptr->player_active[p] &&
            g_ptr->PP[p] == 0) {
            token = g_ptr->PN[p][0];
        }
    }
}
```



```

    }
    printf(" %c |", token);
}
printf(" Start (R0)\n");
printf("-----\n");
// 4. Player Positions Table
printf("\n Current Standing:\n");
for (int i = 0; i < MP; i++) {
    if (g_ptr->player_active[i]) {
        printf(" %-10s | R%-2d\n",
            g_ptr->PN[i],
            g_ptr->PP[i]);
    }
}
// 5. Messages
printf("\n-----\n");
if (last_action && strlen(last_action) > 0) printf(">> %s\n", last_action);
    if (current_status) printf(">> %s\n", current_status);
}

void play_game() {
    char fifo_write[256], fifo_read[256];
    snprintf(fifo_write, sizeof(fifo_write), "%s%d_to_server", FIFO_PREFIX, player_id);
    snprintf(fifo_read, sizeof(fifo_read), "%s%d_from_server", FIFO_PREFIX, player_id);

    int fd_write = open(fifo_write, O_WRONLY);
    int fd_read = open(fifo_read, O_RDONLY);

    if (fd_write == -1 || fd_read == -1) return;

    srand(time(NULL) ^ getpid());

    char my_last_action[256] = "";
    char current_status[256];

    while (g_ptr->game_active)
    {
        if (g_ptr->CT != player_id)
        {
            snprintf(current_status, sizeof(current_status), "Waiting for %s...",
                g_ptr->PN[g_ptr->CT]);

```

```

        d_grid(my_last_action, current_status);
        sleep(1);
        continue;
    }
    d_grid(my_last_action, "YOUR TURN! Press ENTER to roll...");
    getchar();

```

```

char buffer[256] = "ROLL";
write(fd_write, buffer, strlen(buffer) + 1);

```

```

memset(buffer, 0, sizeof(buffer));
if (read(fd_read, buffer, sizeof(buffer)) > 0) {
    int roll;
    sscanf(buffer, "ROLLED %d", &roll);

```

```

    snprintf(my_last_action, sizeof(my_last_action),
        "You rolled a %d! Moved to R%d.",
        roll, g_ptr->PP[player_id]);

```

```

    d_grid(my_last_action, "Turn complete.");
    sleep(2);
}
}
close(fd_write);
close(fd_read);
}

```

```

void cleanup() {
    if (g_ptr != NULL) munmap(g_ptr, sizeof(GameInfo));
    char fifo_name[256];
    snprintf(fifo_name, sizeof(fifo_name), "%s%d_to_server", FIFO_PREFIX, player_id);
    unlink(fifo_name);
    snprintf(fifo_name, sizeof(fifo_name), "%s%d_from_server", FIFO_PREFIX, player_id);
    unlink(fifo_name);
}

```